📊 CATEGORY 7: Performance & Observability — Complete Deep Dive

---

7.1 `vmstat` — Memory, Swap, CPU Overview

🔷 What it is in simple terms

`vmstat` (**v**irtual **m**emory **stat**istics) gives you a **single-line snapshot of the entire system's health**: processes, memory, swap, I/O, and CPU — all in one glance. It's the fastest way to answer "what is this machine doing right now?"

🔷 How vmstat works internally

```
vmstat reads from:
  /proc/meminfo     → memory statistics
  /proc/stat        → CPU statistics
  /proc/vmstat      → virtual memory events
  /proc/diskstats   → I/O statistics

Unlike top/htop, vmstat is lightweight:
  - No ncurses overhead
  - No per-process scanning
  - Single read of /proc files
  - Ideal for scripts and quick checks
```

🔷 Core usage and output

```
# One-time snapshot (averages since boot — less useful)
vmstat

# THE standard usage: update every 2 seconds, skip first line
vmstat 2
# The FIRST line = averages since boot (ignore it)
# Lines 2+ = since the previous interval (this is what you want)

# vmstat output:
vmstat 2
# procs  --------memory--------  --swap--  ---io---  --system--  ------
cpu-----
#  r  b   swpd   free  buff cache   si   so   bi   bo   in   cs  us sy  id
wa st
#  2  0      0 1.1G  98M  9.8G    0    0   24    8  234 567   5  2  91   2
```

```
0
#  4  1      0 0.9G  98M  9.8G    0    0    0 4096  890 1234  12  8  68
12  0
#  1  0      0 1.0G  98M  9.8G    0    0    0  512  456  789   8  3  87
2  0

# ── Column-by-column explanation ──────────────────────────────────────

# PROCS:
# r  → run queue: processes WAITING for CPU (not sleeping — ready to run)
#      r > number_of_CPUs = CPU saturation!
# b  → blocked: processes in uninterruptible sleep (waiting for I/O)
#      b > 0 consistently = I/O bottleneck

# MEMORY (KB by default, -S m for MB):
# swpd  → virtual memory used (swap) — non-zero = memory pressure!
# free  → idle memory (not cache, not used)
# buff  → buffer cache (filesystem metadata)
# cache → page cache (file data) — this is GOOD, reclaimable

# SWAP:
# si → swap in: pages read FROM swap to RAM (memory was under pressure)
# so → swap out: pages written TO swap FROM RAM
#      si/so > 0 = system is/was under memory pressure RIGHT NOW

# IO:
# bi → blocks in: disk reads (blocks/second)
# bo → blocks out: disk writes (blocks/second)

# SYSTEM:
# in → interrupts per second (hardware: disk, NIC, timer)
# cs → context switches per second
#      Very high cs = many short tasks, possible scheduling overhead

# CPU (percentages):
# us → user space CPU (your application code)
# sy → kernel/system CPU (syscalls, kernel processing)
# id → idle CPU (doing nothing)
# wa → iowait: CPU idle but waiting for I/O to complete
#      wa > 10% consistently = I/O bottleneck
# st → stolen: hypervisor stealing CPU from this VM (cloud VMs!)
#      st > 0 = noisy neighbor problem in cloud
```

🔷 Reading the story from vmstat

```
# ── Scenario 1: CPU-bound ──────────────────────────────────────────────
#  r  b  swpd  free  buff  cache  si  so   bi    bo   in    cs  us  sy  id
wa
# 16  0     0   2G   98M   8.9G   0   0    0     8  1234  567  95   3   2
0
```

```
#  ↑                                                              ↑        ↑
# r=16 (many waiting for CPU)                              us=95%  id=2%
# Diagnosis: CPU is the bottleneck, CPUs are fully used

# ── Scenario 2: I/O-bound ──────────────────────────────────────────────
#  r  b  swpd  free  buff  cache  si  so    bi      bo    in    cs  us  sy
id  wa
#  1  8     0   2G   98M   8.9G   0   0     0    98304   890   234   5   8
15  72
#     ↑                                        ↑
↑
# b=8 (blocked on I/O)                       bo=96MB/s write
wa=72%
# Diagnosis: I/O bottleneck, disk write saturated

# ── Scenario 3: Memory pressure (swapping) ────────────────────────────
#  r  b  swpd   free  buff  cache   si   so    bi    bo   in   cs  us  sy
id  wa
#  2  0 2048M  50M    2M    100M   512  1024   512   1024  890  234  40
30  25   5
#              ↑    ↑ ↑    ↑      ↑    ↑
#           swpd=2GB used   very little free/cache  si+so>0 = SWAPPING!
# Diagnosis: serious memory pressure, system is swapping heavily

# ── Scenario 4: Healthy system ────────────────────────────────────────
#  r  b  swpd   free   buff  cache  si  so    bi    bo   in    cs  us  sy
id  wa
#  1  0     0   1.1G   98M   9.8G   0   0    24     8  234   567   5   2
91   2
# r≤CPUs  b=0  swpd=0       lots of cache    si=so=0       id=91%  wa=2%
# Diagnosis: healthy — light load, no memory pressure, minimal I/O

# ── Scenario 5: Noisy neighbor (cloud VM) ────────────────────────────
#  r  b  swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa
st
#  2  0     0   2G   98M   8.9G   0   0   0   8  234 567  40   5  45   2
10
#
↑
#
st=10% stolen
# Diagnosis: hypervisor taking 10% of CPU — noisy neighbor or throttling
```

🔷 More vmstat modes

```
# Memory statistics in MB
vmstat -S M 2

# Show active/inactive memory breakdown
vmstat -a 2
```

```
# --memory-- becomes: swpd free inact active

# Disk statistics (per device, not block-level)
vmstat -d

# Partition statistics
vmstat -p /dev/sda1

# Show event counters (faults, swaps, etc.)
vmstat -s
# 16148936 K total memory
#  4234567 K used memory
#  5234876 K active memory
#  7654320 K inactive memory
#  1124820 K free memory
#    98524 K buffer memory
#  9876234 K swap cache
#        0 K total swap
#        0 K used swap
#        0 K free swap
#   123456 non-nice user cpu ticks
#      456 nice user cpu ticks
#    23456 system cpu ticks
# 12345678 idle cpu ticks
#   234567 IO-wait cpu ticks
#     1234 IRQ cpu ticks
#      567 softirq cpu ticks
#  3456789 pages paged in
#  7654321 pages paged out
#        0 pages swapped in
#        0 pages swapped out
# 45678901 interrupts
# 56789012 CPU context switches
#  1710000000 boot time
#    234567 forks

# Timestamp every line (useful when logging to file)
vmstat -t 2
```

🔷 Production patterns

```
# Capture vmstat to file during incident
vmstat 1 > /tmp/vmstat_$(date +%Y%m%d_%H%M%S).log &
VMSTAT_PID=$!
# ... reproduce the issue ...
kill $VMSTAT_PID

# Quick system health check in script
check_system_health() {
    local line
```

```
    # Get one interval of vmstat (skip the header and first boot-avg line)
    line=$(vmstat 1 2 | tail -1)

    local r b swpd si so wa st
    read -r r b swpd _ _ _ si so _ _ _ _ _ _ _ wa st <<< "$line"

    echo "Run queue: $r | Blocked: $b | Swap used: ${swpd}KB"
    echo "Swap in: ${si} | Swap out: ${so} | IO wait: ${wa}%"
    [[ "$r" -gt 8 ]]  && echo "⚠  CPU SATURATED: r=$r"
    [[ "$b" -gt 4 ]]  && echo "⚠  IO BOTTLENECK: b=$b"
    [[ "$si" -gt 0 || "$so" -gt 0 ]] && echo "🚨 SWAPPING: si=$si so=$so"
    [[ "$wa" -gt 20 ]] && echo "⚠  HIGH IOWAIT: wa=${wa}%"
}
```

🔷 Short crisp interview answer

> "`vmstat 2` is my first command on any system showing performance issues — it gives a bird's-eye view of processes, memory, I/O, and CPU in one line every 2 seconds. I ignore the first line (it's a boot average). Key signals: `r` > number of CPUs means CPU saturation; `b` > 0 persistently means I/O blocking; `si`/`so` > 0 means the system is actively swapping (memory pressure emergency); `wa` > 10% means I/O is making CPUs wait; `st` > 0 on a cloud VM means the hypervisor is stealing CPU time."

⚠ Gotchas

```
# GOTCHA 1: First vmstat line is since-boot average — misleading!
vmstat       # Shows averages since boot
vmstat 2     # Lines after first = last 2 seconds — USE THESE

# GOTCHA 2: r (run queue) counts include the process currently running
# On a 4-CPU system:  r=4 is normal (one process per CPU)
# On a 4-CPU system:  r=16 means 12 extra processes waiting = saturated

# GOTCHA 3: wa (iowait) is percentage of CPU time
# wa=50% means "50% of CPU time, CPUs had nothing to do except wait for
I/O"
# NOT "50% of I/O capacity used"
# wa can be 0% even with disk at 100% if other work keeps CPUs busy

# GOTCHA 4: swpd is cumulative, not current pressure
# swpd=2GB means 2GB is currently IN swap
# si=0, so=0 with swpd=2GB = swap was used in past, not active now
# The DANGEROUS signal is si/so > 0 (active swapping right now)

# GOTCHA 5: vmstat units are blocks (512 bytes) for bi/bo
# bi=1000 = 512,000 bytes/sec = 500 KB/sec (not 1MB/sec)
# Use vmstat -S M for MB-based memory, but bi/bo stay in blocks
```

7.2 `mpstat`, `iostat`, `sar` — The `sysstat` Suite

🔷 What sysstat is

The `sysstat` package is a collection of performance monitoring tools that all share a common format and can record historical data. The key members: `mpstat` (per-CPU stats), `iostat` (I/O stats), and `sar` (system activity recorder — the historian).

```
# Install
sudo apt install sysstat
sudo yum install sysstat

# Enable historical collection (crucial!)
sudo systemctl enable --now sysstat
# This runs sadc (data collector) every 10 minutes
# Stores data in /var/log/sysstat/sa<day>
```

🔷 `mpstat` — Per-CPU breakdown

```
# ── Why mpstat over vmstat? ──────────────────────────────────────────
# vmstat shows combined CPU usage — hides per-core imbalances
# mpstat shows EACH core individually — reveals:
#   - Single-threaded bottleneck (one core at 100%, others idle)
#   - IRQ affinity issues (one core handling all interrupts)
#   - NUMA imbalances

# All CPUs, every 2 seconds
mpstat -P ALL 2

# Output:
# Linux 5.15.0 (server01)   03/10/2026  _x86_64_  (8 CPU)
#
# 14:32:01  CPU    %usr  %nice  %sys  %iowait  %irq  %soft  %steal  %guest
%idle
# 14:32:03  all    45.3   0.0   8.2     2.1    0.1    0.5     0.0     0.0
43.8  ← aggregate
# 14:32:03    0    95.0   0.0   4.5     0.5    0.0    0.5     0.0     0.0
0.0  ← CPU 0: 100%!
# 14:32:03    1     2.0   0.0   1.0     3.5    0.0    0.5     0.0     0.0
93.0  ← CPU 1: idle
# 14:32:03    2     2.0   0.0   1.0     3.0    0.0    0.5     0.0     0.0
93.5
# 14:32:03    3     1.5   0.0   1.0     4.0    0.2    0.5     0.0     0.0
92.8
# ... (CPUs 4-7 also mostly idle)
```

```
# ↑ This pattern reveals: single-threaded bottleneck on CPU 0
#   vmstat would show: all=45% → looks fine, hides the real problem!

# Specific CPU only
mpstat -P 0 2            # Only CPU 0

# Summary with interrupts
mpstat -I ALL 2          # Show interrupt statistics per CPU

# Column explanations:
# %usr    → user space (application code, no kernel)
# %nice   → user space with reduced priority
# %sys    → kernel/system (syscalls, kernel threads)
# %iowait → idle but waiting for I/O (same as vmstat wa)
# %irq    → hardware interrupt handling
# %soft   → software interrupt (network stack, etc.)
# %steal  → hypervisor steal time (cloud VMs)
# %guest  → running virtual CPU for guest (hypervisor host)
# %idle   → truly doing nothing
```

🔷 `iostat` — I/O statistics (deep review)

```
# Already covered in Category 6, but key sysstat-specific usage:

# Extended stats every 2 seconds, human readable
iostat -hx 2

# Show only specific device
iostat -hx sda nvme0n1 2

# First line skipping (same as vmstat — first = since boot)
iostat -hx 2 | grep -v "^$" | awk 'NR>3'  # Skip header + first report

# Key metrics recap:
# %util     → device busy time (>95% = saturated for HDD)
# r_await   → read latency in ms (SSD: <1ms, HDD: <10ms)
# w_await   → write latency in ms
# aqu-sz    → average queue depth (>1 persistently = saturated)
# rrqm/s    → read requests merged (adjacent requests combined =
efficiency)
# wrqm/s    → write requests merged
```

🔷 `sar` — The historical recorder (most powerful)

```
# sar = System Activity Reporter
# Records everything: CPU, memory, I/O, network, swap, context switches
```

```
# ── Real-time (like vmstat/iostat) ──────────────────────────────

sar 2 5           # CPU stats: every 2 seconds, 5 reports
sar -r 2          # Memory stats every 2 seconds
sar -b 2          # I/O transfer stats every 2 seconds
sar -n DEV 2      # Network interface stats every 2 seconds
sar -u ALL 2      # Per-CPU stats (like mpstat)

# ── Historical data — THE killer feature ────────────────────────

# View today's CPU stats (from background sadc collection)
sar -u
# 12:00:01 AM  CPU   %user  %nice  %system  %iowait  %steal  %idle
# 12:10:01 AM  all    5.23   0.00     1.45     0.12    0.00   93.20
# 12:20:01 AM  all    8.45   0.00     2.12     0.34    0.00   89.09
# 12:30:01 AM  all   45.67   0.00    12.34     8.90    0.00   33.09  ←
spike!
# ...
# Average:     all   12.45   0.00     3.21     1.23    0.00   83.11

# View yesterday's data
sar -u -f /var/log/sysstat/sa09    # Day 9 of current month

# View specific date
sar -u -f /var/log/sysstat/sa$(date -d "yesterday" +%d)

# ── What sar can show historically ──────────────────────────────

sar -u           # CPU utilization
sar -r           # Memory utilization (free, buffers, cache)
sar -r ALL       # Extended memory (huge pages, committed, etc.)
sar -b           # I/O transfer rates (tps, bread, bwrtn)
sar -d           # Per-device I/O (like iostat)
sar -n DEV       # Network interface stats (rxpck/s, txpck/s, rxkB/s,
txkB/s)
sar -n EDEV      # Network errors
sar -n SOCK      # Socket statistics
sar -n TCP       # TCP stats (active/passive opens, retransmits)
sar -q           # Load average and run queue
sar -S           # Swap utilization
sar -W           # Swapping statistics (pswpin/s, pswpout/s)
sar -v           # Kernel table stats (inodes, files, sockets)
sar -w           # Context switches and process creation

# ── Historical investigation workflow ───────────────────────────

# "The system was slow yesterday at 2PM — what happened?"

# Step 1: CPU — was it CPU-bound?
sar -u -f /var/log/sysstat/sa09 | grep "02:0"
# 02:00:01 PM  all   89.45   0.00    8.12     0.34    0.00    2.09  ← CPU
spike!

# Step 2: Memory — was it swapping?
```

```
sar -W -f /var/log/sysstat/sa09 | grep "02:0"
# 02:00:01 PM  pswpin/s  pswpout/s
# 02:00:01 PM     45.23      23.45  ← yes, swapping heavily!

# Step 3: I/O — was the disk saturated?
sar -b -f /var/log/sysstat/sa09 | grep "02:0"

# Step 4: Network — was there a traffic spike?
sar -n DEV -f /var/log/sysstat/sa09 | grep "02:0"

# ── sar output: memory (sar -r) ──────────────────────────────────────
sar -r 2
# Time     kbmemfree kbavail kbmemused %memused kbbuffers  kbcached
kbcommit %commit kbactive kbinact kbdirty
# 14:30:01  1124820 10643916  5223456    32.34    98524    9876234
3456789   21.40  5234876 7654320  123456
#                   ↑
↑
#                kbavail = truly available
kbdirty = write buffer

# ── JSON output for monitoring pipelines ──────────────────────────────
sar -u 1 1 --json | python3 -c "
import sys, json
data = json.load(sys.stdin)
hosts = data['sysstat']['hosts'][0]
stats = hosts['statistics'][0]['cpu-load'][0]
print(f\"CPU: user={stats['user']}% sys={stats['system']}% idle=
{stats['idle']}%\")
"
```

🔷 Short crisp interview answer

> "The sysstat suite — `mpstat`, `iostat`, `sar` — are my tools for both real-time and historical performance analysis. `mpstat -P ALL 2` shows per-CPU breakdown, which is critical because `vmstat` hides single-threaded bottlenecks where one core is at 100% while others are idle. `sar` is the most powerful — it runs as a background daemon collecting data every 10 minutes, so I can run `sar -u -f /var/log/sysstat/sa<day>` to see what the system was doing days ago during an incident, instead of guessing."

7.3 `free` — Memory, Buffers, Cache — What's Actually "Free" ⚠️

🔷 The most misunderstood command in Linux

This section has a warning flag for a reason. `free` output confuses even experienced engineers. The key insight: **Linux intentionally uses all available RAM for caching, and this is correct behavior, not a**

problem.

🔷 How Linux memory management works

```
RAM is divided into:

┌──────────────────────────────────────────────────────────┐
│  Kernel reserved (non-reclaimable)                       │
├──────────────────────────────────────────────────────────┤
│  Process memory (heap, stack, text) — anonymous pages    │
│  Cannot be reclaimed without swapping                    │
├──────────────────────────────────────────────────────────┤
│  Page cache (file data) — reclaimable                    │
│  Kernel drops this when processes need memory            │
│  ← Linux fills this with recently read files             │
├──────────────────────────────────────────────────────────┤
│  Buffer cache (fs metadata, block device data)           │
│  Also reclaimable                                        │
├──────────────────────────────────────────────────────────┤
│  Free (truly unused) ← Linux tries to minimize this      │
│  Unused RAM is wasted RAM                                │
└──────────────────────────────────────────────────────────┘

The caching is why:
  First access: read from disk (slow)
  Second access: read from page cache (RAM speed)
  Performance gain: 1000× faster
```

🔷 Reading `free` correctly

```
free -h
#               total        used        free      shared  buff/cache
available
# Mem:          15.4G        4.2G      512.0M      234.0M       10.6G
10.7G
# Swap:          2.0G          0B        2.0G

# ── Column meanings ──────────────────────────────────────────

# total      → physical RAM installed
#              15.4G in this example

# used       → RAM consumed by processes + kernel
#              4.2G — this is actual application memory

# free       → RAM with NO use AT ALL (not cache, not used)
#              512MB — looks scary! But...

# shared     → RAM used by tmpfs (shared memory)
```

```
#              234MB — shared between processes

# buff/cache → page cache + buffer cache
#              10.6G — this is RECLAIMABLE (Linux will drop it if needed)

# available  → THE NUMBER THAT ACTUALLY MATTERS
#              = free + buff/cache (reclaimable portion)
#              10.7G — this is what new processes CAN use

# ── The wrong way to read free ──────────────────────────────────

free -h
# free = 512MB → WRONG interpretation: "only 512MB left, system is full!"
#                This causes panic and unnecessary server restarts

# ── The right way to read free ──────────────────────────────────

free -h
# available = 10.7G → CORRECT: "10.7GB available for new processes"
#                     System is completely fine

# ── The ACTUAL danger signs ──────────────────────────────────

free -h
#               total   used    free  shared  buff/cache  available
# Mem:          15.4G  15.1G  50.0M   234M      200M        80.0M  ←
DANGER!
# Swap:          2.0G   1.8G   200M

# available = 80MB ← Very low! Memory pressure
# Swap used = 1.8GB ← System is actively using swap
# This is genuinely concerning

# ── Free output: what the old format showed ──────────────────

# Old free (pre-3.3) output:
#              total       used       free     shared    buffers
cached
# Mem:         15400      14800        600        234        100
9800
# -/+ buffers/cache:       4900      10500  ← this line: apps
used/available
# Swap:         2000          0       2000

# The "-/+ buffers/cache" row was the "true" used/free
# Modern free shows "available" column instead (better)
```

🔷 Memory deep dive with `/proc/meminfo`

```
cat /proc/meminfo
# MemTotal:      16148936 kB   ← total physical RAM
# MemFree:        1124820 kB   ← truly unused
# MemAvailable:  10643916 kB   ← available to start new apps (KEY METRIC)
# Buffers:          98524 kB   ← block device buffer cache
# Cached:         9876234 kB   ← page cache (file contents in RAM)
# SwapCached:           0 kB   ← pages in swap that are also in RAM
# Active:         5234876 kB   ← recently used pages (less likely to
reclaim)
# Inactive:       7654320 kB   ← older pages (candidates for reclaim)
# Active(anon):   2345678 kB   ← active process memory (heap/stack)
# Inactive(anon): 1234567 kB   ← old process memory (swap candidate)
# Active(file):   2889198 kB   ← active file cache
# Inactive(file): 6419753 kB   ← inactive file cache (first to be
reclaimed)
# Mlocked:          12345 kB   ← pages locked in RAM (mlock() — cannot be
swapped)
# SwapTotal:      2097148 kB   ← total swap space
# SwapFree:       2097148 kB   ← swap space available
# Dirty:           123456 kB   ← modified pages not yet written to disk
# Writeback:         1234 kB   ← pages being written to disk right now
# AnonPages:      3456789 kB   ← anonymous (not file-backed) pages
# Mapped:          234567 kB   ← memory-mapped files
# Shmem:           234098 kB   ← tmpfs + SysV shared memory
# KReclaimable:  1234567 kB   ← kernel memory that can be reclaimed
# Slab:          1456789 kB   ← kernel slab allocator memory
# SReclaimable:  1234567 kB   ← slab that can be reclaimed (caches)
# SUnreclaim:     222222 kB   ← slab that cannot be reclaimed
# VmallocTotal:  34359738367 kB  ← virtual memory available
# HugePages_Total:     0       ← huge pages configured
# HugePages_Free:      0

# ── What to watch for memory pressure ──────────────────────────────

# 1. MemAvailable dropping toward 0 → applications will start failing
# 2. SwapFree decreasing → kernel is moving memory to swap (slow!)
# 3. Dirty growing very large → disk can't keep up with writeback
# 4. Writeback staying non-zero → disk write bottleneck
# 5. SUnreclaim growing → kernel slab leak (kernel bug or
misconfiguration)
```

🔷 Memory usage breakdown by process

```
# Which processes use the most memory?

# RSS (physical RAM in use)
ps aux --sort=-%mem | head -10
ps -eo pid,ppid,cmd,%mem,rss --sort=-rss | head -10
```

```
# smem — more accurate (counts shared pages proportionally)
sudo smem -rs rss | head -10
# Process           PID  USS    PSS    RSS
# java             1234  2.1G  2.1G   2.2G   ← USS=unique,
PSS=proportional, RSS=physical

# /proc/<PID>/status
cat /proc/1234/status | grep -E "VmRSS|VmPeak|VmSwap"
# VmPeak:  2345678 kB   ← peak virtual memory used
# VmRSS:   2123456 kB   ← current physical RAM used
# VmSwap:       0 kB   ← memory in swap

# /proc/<PID>/smaps_rollup — most accurate per-process breakdown
cat /proc/1234/smaps_rollup
# Rss:             2123456 kB
# Pss:             2034567 kB   ← proportional share (shared pages
divided)
# Shared_Clean:      89012 kB   ← clean shared (like shared libs)
# Private_Clean:    456789 kB   ← private clean (can be dropped)
# Private_Dirty:   1567655 kB   ← private dirty (CANNOT be dropped without
swap)
```

🔷 Short crisp interview answer

> "`free -h` is commonly misread. The `free` column looks alarming when low, but `available` is the correct metric — it includes reclaimable page cache. Linux intentionally fills unused RAM with file cache because unused RAM is wasted RAM. The real danger signals are: `available` approaching 0, non-zero swap used actively (`SwapFree` decreasing over time), and in `/proc/meminfo`, `Dirty` growing very large (disk can't keep up). I always check `available`, not `free`, when assessing memory health."

⚠ Gotchas

```
# GOTCHA 1: "My server has 16GB RAM and 15GB used — it's out of memory!"
# Almost always wrong. Check available:
free -h | awk '/Mem:/ {print "Available:", $7}'
# If available > 1GB, the system is fine

# GOTCHA 2: High buffers/cache is NOT a memory leak
# Linux holds cache as long as nothing else needs the RAM
# Applications needing memory? Kernel drops cache immediately
# Cache growing = more files being read = expected behavior

# GOTCHA 3: Swap used ≠ swap being used RIGHT NOW
free -h
# Swap: 2.0G   200M   1.8G  ← 200MB used — but is it actively swapping?
vmstat 1 | awk '{print $7, $8}'   # si, so columns
# si=0, so=0 → swap is used but NOT actively swapping (old data parked
```

```
there)
# si>0 or so>0 → actively swapping → REAL memory pressure

# GOTCHA 4: OOM killer fires even with "available" memory
# Kernel may OOM kill if:
# - vm.overcommit_memory setting forbids the allocation
# - A cgroup memory limit is hit (container limits)
# - Single allocation is too large (fragmentation)
# Check: dmesg | grep -i "oom"

# GOTCHA 5: Hugepages removed from "free" reporting
# If HugePages configured, memory reserved for hugepages doesn't appear
# in MemFree — looks like memory disappeared
cat /proc/meminfo | grep HugePages
```

7.4 Load Average — What 1/5/15 Minute Values Mean ⚠️

🔷 What load average is (and what it is NOT)

Load average is the **average number of processes in a runnable or uninterruptible state** over the last 1, 5, and 15 minutes. It is NOT CPU utilization percentage. This distinction trips up almost everyone.

🔷 The precise definition

```
Load average counts processes that are:
  1. Running on CPU right now (RUNNING state — R)
  2. Waiting to run on CPU (RUNNABLE state — R)
  3. In uninterruptible sleep — waiting for I/O (D state)

It does NOT count:
  - Sleeping processes waiting for events (S state) — normal sleep
  - Stopped processes (T state)
  - Zombie processes (Z state)

So load average = CPU demand + I/O demand combined
```

🔷 How to interpret the three numbers

```
uptime
# 14:32:01 up 42 days, 3:15,  2 users,  load average: 2.45, 1.89, 1.23
#                                                      ────  ────  ────
#                                                      1min  5min  15min
```

```
# The question is: load relative to WHAT?
# Answer: relative to the number of CPUs

# ── Rule of thumb ──────────────────────────────────────────────────
# Perfect load on an N-CPU system = N (one job per CPU, no waiting)
# load < N  → system has spare capacity
# load = N  → system is exactly saturated (no queue)
# load > N  → processes are waiting → system is overloaded

# ── Example: 4 CPU system ──────────────────────────────────────────
# load: 4.00 → exactly saturated (normal, no headroom)
# load: 2.00 → 50% utilized, healthy
# load: 8.00 → 2× overloaded, 4 processes always waiting for CPU
# load: 0.25 → lightly loaded, plenty of spare capacity

# Number of CPUs on this system:
nproc                     # logical CPUs (includes hyperthreading)
nproc --all               # same
grep -c processor /proc/cpuinfo

# Per-CPU load calculation:
python3 -c "
import os
load = os.getloadavg()
cpus = os.cpu_count()
print(f'Load 1min: {load[0]:.2f} | Per CPU: {load[0]/cpus:.2f}')
print(f'Load 5min: {load[1]:.2f} | Per CPU: {load[1]/cpus:.2f}')
print(f'Load 15min: {load[2]:.2f} | Per CPU: {load[2]/cpus:.2f}')
print(f'Status: {\"SATURATED\" if load[0]/cpus > 1 else \"OK\"}')
"
```

🔷 Reading the trend

```
# The three numbers tell a STORY about direction:

# load: 8.50, 4.00, 1.50  → INCREASING: something just started/spiked
#       ────  ────  ────
#       1min  5min  15min
# 1min >> 15min: recent spike, getting worse

# load: 1.50, 4.00, 8.50  → DECREASING: was bad, improving now
# 1min << 15min: was a problem, now recovering

# load: 4.10, 4.05, 4.02  → STABLE: sustained moderate load
# All three similar: steady state

# load: 0.50, 0.49, 0.51  → FLAT LOW: lightly loaded, healthy
```

🔷 Load average ≠ CPU utilization

```
# CRITICAL DISTINCTION:
# A disk-heavy workload with few CPUs can cause high load average
# while CPU utilization is LOW.

# Example: database doing heavy I/O
# 16 processes blocked waiting for disk (D state)
# 0 processes actually running on CPU
# CPU utilization: 0%
# Load average: 16.00 (all those D-state processes counted!)

# How to distinguish CPU load vs I/O load:
vmstat 1
#  r  b   ...  wa
#  0 16   ...  95   → b=16 (blocked on I/O), wa=95% → I/O load, not CPU

# vs CPU load:
vmstat 1
# r  b   ...  wa
# 16  0  ...  0    → r=16 (waiting for CPU), wa=0% → CPU load

# ── Load average on multicore systems ──────────────────────────────

# 32-core server with load 16.00:
#   16/32 = 0.5 per CPU → healthy, 50% utilized

# 2-core laptop with load 16.00:
#   16/2 = 8.0 per CPU → severely overloaded!

# Same load average, VERY different situations!
# ALWAYS normalize by CPU count

# ── Linux-specific: D-state processes inflate load ────────────────
# Linux counts D-state (uninterruptible sleep) in load average
# Other UNIX systems (BSDs, Solaris) traditionally did NOT
# This means Linux load average includes I/O pressure
# High load with low CPU usage → I/O bottleneck (check vmstat b column)
```

🔷 Practical thresholds

```
# Per-CPU load (load_1min / nproc):

# 0.0 - 0.7   → Ideal. System has plenty of headroom.
# 0.7 - 1.0   → Comfortable. Monitor trends.
# 1.0         → Fully utilized but just at capacity. No queue forming yet.
# 1.0 - 1.5   → Mild saturation. Processes occasionally waiting.
# 1.5 - 3.0   → Moderate saturation. Noticeable latency increase.
# 3.0+        → Heavy saturation. Serious queuing. Investigate
```

```
immediately.
# 10.0+       → Crisis. System may become unresponsive.

# Script: alert on load
check_load() {
    local cpus threshold load_1min per_cpu
    cpus=$(nproc)
    threshold=1.5   # Alert if per-CPU load > 1.5
    load_1min=$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1
| tr -d ' ')
    per_cpu=$(echo "scale=2; $load_1min / $cpus" | bc)

    echo "Load 1min: $load_1min | CPUs: $cpus | Per CPU: $per_cpu"
    if (( $(echo "$per_cpu > $threshold" | bc -l) )); then
        echo "⚠  HIGH LOAD: per-CPU load $per_cpu > $threshold"
        return 1
    fi
    return 0
}
```

🔷 Short crisp interview answer

> "Load average is the average number of processes in runnable or uninterruptible (D) state over 1, 5, and 15 minutes — it's NOT CPU utilization. The key is to divide by CPU count: on a 16-core system, a load of 16 is fine (1.0 per CPU), but on a 2-core system it's 8× overloaded. The three numbers show the trend: if 1min >> 15min, it's a recent spike getting worse; if 1min << 15min, it was worse and is recovering. Linux includes D-state (I/O-waiting) processes in load average, so high load with low CPU% in vmstat means I/O is the bottleneck, not CPU."

⚠ Gotchas

```
# GOTCHA 1: High load on idle-looking CPU
top      # CPU: 2% used, load: 15.00
# Explanation: 15 processes blocked in D state waiting for disk
# They count in load average but not CPU %
# Check: vmstat 1 (look at b column)

# GOTCHA 2: Load average is an exponential moving average
# It's NOT a strict 1-minute average of processes
# It's a EWMA that weights recent samples more
# So load doesn't respond instantly to changes — takes ~5 minutes to
reflect

# GOTCHA 3: Hyperthreading inflates "CPU count"
nproc   # Shows logical CPUs including hyperthreads
# 32 logical CPUs might be 16 physical cores × 2 hyperthreads
# Performance scales roughly 1.3× per physical core, not 2×
# Some engineers use physical cores for load threshold calculations
```

```
# GOTCHA 4: Load > 1.0 per CPU isn't always bad
# Short spikes above 1.0 are normal (momentary bursts)
# Problem: sustained load > 1.0 per CPU over 5+ minutes

# GOTCHA 5: Load average in containers/cgroups
# Inside a container: load average reflects HOST system, not container
# A container on a 64-core host will show host's load, even if limited to
2 CPUs
# This is a known limitation — use container-specific metrics instead
```

7.5 `perf` Basics — CPU Profiling, Flamegraphs

🔷 What `perf` is

`perf` is the **Linux profiling powerhouse** — it uses hardware performance counters and kernel tracepoints to measure with near-zero overhead what the CPU is actually doing. It can find where a program spends its time, which functions are called most, and what hardware events (cache misses, branch mispredictions) are occurring.

🔷 How perf works internally

```
Hardware Performance Counters:
  Modern CPUs have 4-8 hardware counters that count:
  - CPU cycles, instructions retired
  - Cache hits/misses (L1, L2, L3)
  - Branch predictions/mispredictions
  - TLB misses

perf uses two modes:
  1. Counting: count events over a time period (cheap)
  2. Sampling: sample the call stack at N events/second
     (statistical — catches where time is spent)

Sampling mechanism:
  Every N cycles → CPU interrupt → kernel captures:
    - Which process
    - Instruction pointer (which function)
    - Call stack (how we got here)
  → Aggregate → flame graph → visual CPU profile
```

🔷 Core `perf` usage

```
# Install
sudo apt install linux-tools-common linux-tools-$(uname -r)

# ── perf stat — count hardware events ──────────────────────────────

# Run a command and count CPU events
perf stat ls -la /

# Output:
# Performance counter stats for 'ls -la /':
#      1.23 msec task-clock          # 1.23ms of CPU time used
#         0      context-switches    # no context switches
#         0      cpu-migrations      # stayed on same CPU
#       156      page-faults         # 156 page faults
#   4,234,567    cycles              # CPU cycles consumed
#   6,789,012    instructions        # instructions retired
#         1.60   insn per cycle      # IPC: higher = more efficient
#     345,678    branches
#      12,345    branch-misses       # 3.6% miss rate
#       89,012   cache-misses        # L1 cache misses

# Specific events
perf stat -e cycles,instructions,cache-misses ./myprogram

# All available events
perf list

# ── perf top — real-time CPU hotspot finder ────────────────────────

# Which functions are consuming CPU RIGHT NOW (systemwide)
sudo perf top

# Output (like top but for functions):
# Overhead  Shared Object      Symbol
#   45.23%  [kernel]           [k] do_raw_spin_lock
#   12.34%  libc-2.31.so       [.] __memcpy_avx_unaligned
#    8.90%  myapp              [.] process_request
#    5.67%  [kernel]           [k] __x86_indirect_thunk_rax
#    3.45%  libssl.so.1.1      [.] EVP_DigestUpdate

# Specific process only
sudo perf top -p 1234

# ── perf record + report — capture and analyze ─────────────────────

# Record profile of a command
perf record ./myprogram

# Record a running process
sudo perf record -p 1234 sleep 30   # Profile PID 1234 for 30 seconds

# Record with call graph (needed for flamegraph)
sudo perf record -g -p 1234 sleep 30
```

```
# -g = record call stack for each sample

# Record with frequency
sudo perf record -F 999 -g -p 1234 sleep 30
# -F 999 = 999 samples per second

# View report
perf report

# Output:
# Overhead  Command   Shared Object     Symbol
#   42.34%  myapp     myapp             [.] hash_lookup
#   18.90%  myapp     libc-2.31.so      [.] malloc
#   12.45%  myapp     myapp             [.] process_data
#    8.34%  myapp     [kernel]          [k] copy_user_enhanced_fast_string

# Annotate: see which assembly instructions are hot
perf annotate hash_lookup
```

🔷 Flamegraphs — The visualization that changed profiling

```
# Flamegraphs visualize the perf record output as a stack visualization
# Each frame = a function
# Width = how much CPU time it consumed
# Height = call depth

# ── Generating flamegraphs ──────────────────────────────────────

# Get Brendan Gregg's flamegraph tools
git clone https://github.com/brendangregg/FlameGraph /opt/flamegraph

# Step 1: Record with call graph
sudo perf record -F 99 -g -p $(pgrep myapp) sleep 30

# Step 2: Convert to folded format
sudo perf script | /opt/flamegraph/stackcollapse-perf.pl >
/tmp/perf.folded

# Step 3: Generate SVG flamegraph
/opt/flamegraph/flamegraph.pl /tmp/perf.folded > /tmp/flamegraph.svg

# Open in browser — interactive! Click to zoom into call trees

# ── Reading a flamegraph ────────────────────────────────────────
# Y axis = call stack depth (bottom = root, top = leaf functions)
# X axis = sample count (wider = more CPU time) — NOT time ordering
# Colors = random (meaningless) or by library type
# Widest frames = CPU hotspots

# Color coding (Brendan Gregg convention):
```

```
# Red/orange = user-space code
# Yellow = kernel code
# Green = JIT compiled (Java, Node.js)
# Blue = C library

# ── One-liner: CPU flamegraph ──────────────────────────────────────
sudo perf record -F 99 -g -a sleep 30 && \
    perf script | /opt/flamegraph/stackcollapse-perf.pl | \
    /opt/flamegraph/flamegraph.pl > flamegraph.svg

# ── Off-CPU flamegraph (find where time is BLOCKED, not running) ────
# Shows time spent sleeping, waiting for locks, I/O
sudo /opt/bcc/tools/offcputime-bpfcc -df -p $(pgrep myapp) 30 | \
    /opt/flamegraph/flamegraph.pl --color=io --title="Off-CPU Time" \
    --countname=us > offcpu.svg
```

🔷 Specific profiling scenarios

```
# ── Scenario: "My app is using too much CPU" ──────────────────────

# Find the hot function:
sudo perf top -p $(pgrep myapp)
# See: hash_lookup is 45% of CPU → that function is the bottleneck

# Get call stacks to understand why:
sudo perf record -F 99 -g -p $(pgrep myapp) sleep 30
perf report --stdio | head -50

# ── Scenario: "Cache misses are killing performance" ──────────────

perf stat -e L1-dcache-loads,L1-dcache-load-misses,LLC-loads,LLC-load-
misses \
    ./myprogram
# L1-dcache-load-misses = 35%  → very high! Check data access patterns

# ── Scenario: "System calls are slow" ─────────────────────────────

sudo perf trace -p 1234     # Trace syscalls like strace but faster
# Overhead much lower than strace (uses perf's eBPF engine)

# ── Scenario: Hardware performance counter profiling ──────────────

# CPU branch misprediction (causes pipeline stalls)
perf stat -e branch-misses,branches ./myprogram
# branch-misses = 15%  → high! Review conditional branch patterns

# Memory bandwidth
perf stat -e cache-references,cache-misses \
    -e mem-loads,mem-stores ./myprogram
```

🔷 Short crisp interview answer

> "`perf` uses hardware performance counters and kernel tracepoints to profile with minimal overhead. `perf stat` counts events (cycles, instructions, cache misses) for a command. `perf top` shows live CPU hotspots by function systemwide. `perf record -g -F 99` captures call stacks at 99 samples/second, and `perf report` analyzes them. The real power is flamegraphs: pipe `perf script` through Brendan Gregg's `stackcollapse-perf.pl` and `flamegraph.pl` to get an interactive SVG showing exactly which call stacks consume the most CPU. Widest frames in the flamegraph are the bottlenecks."

7.6 USE Method — Utilization, Saturation, Errors Framework

🔷 What the USE Method is

The USE Method is a **systematic performance investigation framework** created by Brendan Gregg. Instead of randomly checking metrics, it gives you a checklist: for every resource, check Utilization, Saturation, and Errors. It ensures you don't miss the bottleneck.

🔷 The three metrics defined

```
For every system resource:

UTILIZATION
  Definition: Percentage of time the resource is busy
  High util → resource is heavily loaded
  100% util → resource is saturated (queuing begins)

  Examples:
    CPU utilization: 85% busy, 15% idle
    Disk utilization: %util in iostat
    Memory utilization: used/total

SATURATION
  Definition: Degree to which there is extra work the resource CANNOT
service
  Shows as: queue lengths, wait times, backlog
  Saturation > 0 → resource is a bottleneck (work is waiting!)

  Examples:
    CPU saturation: run queue length (r in vmstat)
    Disk saturation: I/O queue depth (aqu-sz in iostat)
    Memory saturation: swap activity (si/so in vmstat)

ERRORS
```

```
  Definition: Count of error events
  Errors → hardware or software problems
  Often cause performance degradation

  Examples:
    Network errors: ethtool -S eth0 | grep error
    Disk errors: smartctl -a /dev/sda
    Memory errors: /sys/devices/system/edac/mc*/
```

🔷 USE Method checklist

```
# ── CPU ──────────────────────────────────────────────────────────

# Utilization
mpstat -P ALL 1 1 | awk '/Average/ {print "CPU", $2, "util:", 100-$NF"%"}'
# Or: vmstat 1 1 | tail -1 | awk '{print "CPU idle:", $15"%"}'

# Saturation
vmstat 1 1 | tail -1 | awk '{print "Run queue:", $1}'
# r > nproc → CPU saturated

# Errors
# CPU errors are rare (hardware) — check:
dmesg | grep -i "machine check\|mce\|cpu error"

# ── Memory ───────────────────────────────────────────────────────

# Utilization
free -h | awk '/Mem:/ {print "Memory used:", $3, "/ total:", $2}'
awk '/MemTotal|MemAvailable/ {print}' /proc/meminfo

# Saturation (swap activity = memory under pressure)
vmstat 1 1 | tail -1 | awk '{print "Swap in:", $7, "Swap out:", $8}'
# si/so > 0 → memory saturated (actively swapping)
sar -W 1 1   # Paging statistics

# Errors
dmesg | grep -i "memory\|ECC\|EDAC"
ls /sys/devices/system/edac/mc*/  # Memory error counters

# ── Disk I/O ─────────────────────────────────────────────────────

# Utilization
iostat -hx 1 1 | awk '/^[a-z]/ {print $1, "util:", $NF}'
# %util → how busy the disk is

# Saturation
iostat -hx 1 1 | awk '/^[a-z]/ {print $1, "queue:", $9}'
# aqu-sz > 1 → disk queue building → saturated
```

```
# Errors
sudo smartctl -H /dev/sda        # SMART health (Passed/Failed)
sudo smartctl -A /dev/sda | grep -E "Reallocated|Pending|Uncorrectable"
dmesg | grep -i "I/O error\|ata\|sd[a-z]"

# ── Network ──────────────────────────────────────────────────────────

# Utilization
ip -s link show eth0
# RX/TX bytes compared to link speed (1Gbps = 125MB/s)
sar -n DEV 1 1 | grep eth0
# rxkB/s, txkB/s compared to NIC speed

# Saturation
ss -s | grep "TCP:"
# Large number of established connections → may be saturating
# netstat -s | grep "retransmited"   ← retransmits = saturation
cat /proc/net/dev | awk '/eth0/ {print "RX drop:", $5, "TX drop:", $14}'

# Errors
ip -s link show eth0 | grep -A2 "RX\|TX"
ethtool -S eth0 | grep -i "error\|drop\|miss\|fail"

# ── Full USE sweep: quick script ─────────────────────────────────────

use_method_check() {
    echo "=== USE METHOD CHECK ==="
    echo ""
    echo "--- CPU ---"
    echo -n "Utilization: "; mpstat 1 1 | awk '/Average.*all/ {printf
"%.1f%%\n", 100-$NF}'
    echo -n "Saturation:  "; vmstat 1 2 | tail -1 | awk '{printf "run
queue=%s\n", $1}'
    echo    "Errors:      check dmesg for MCE"

    echo ""
    echo "--- Memory ---"
    echo -n "Utilization: "; free -h | awk '/Mem:/ {printf "%s / %s\n",
$3, $2}'
    echo -n "Saturation:  "; vmstat 1 2 | tail -1 | awk '{printf "swap
in/out: %s/%s\n", $7, $8}'
    echo    "Errors:      check /sys/devices/system/edac/"

    echo ""
    echo "--- Disk ---"
    for disk in $(lsblk -d -n -o NAME | grep -E "^sd|^nvme"); do
        echo -n "  /dev/$disk utilization: "
        iostat -x 1 1 "$disk" | awk '/^[a-z]/ {printf "%.1f%%\n", $NF}'
    done
    echo -n "Saturation:  "; iostat -x 1 1 | awk '/^[a-z]/ {s+=$9} END
{printf "avg queue=%.2f\n", s}'
    echo    "Errors:      run: sudo smartctl -H /dev/sda"

    echo ""
```

```
    echo "--- Network ---"
    echo "Utilization: "; sar -n DEV 1 1 | awk '/eth0|ens|eno/ {printf "
RX: %.1fMB/s TX: %.1fMB/s\n", $5/1024, $6/1024}'
    echo -n "Saturation: "; ss -s | grep "TCP:"
    echo "Errors:      run: ip -s link show eth0"
}
```

🔷 Short crisp interview answer

> "The USE Method by Brendan Gregg gives a systematic checklist for performance analysis: for every resource (CPU, memory, disk, network, bus), check Utilization (how busy?), Saturation (is work queuing?), and Errors (hardware problems?). Saturation is often the most telling — util can be 70% and seem fine, but if the queue depth is growing, the resource IS the bottleneck. I apply it in order: run `vmstat` and `mpstat` for CPU, `iostat` for disk, `sar -n DEV` for network, then drill into whatever shows saturation > 0."

7.7 `dmesg` — Kernel Ring Buffer, OOM Killer Logs

🔷 What the kernel ring buffer is

The kernel ring buffer is a **fixed-size circular buffer** in kernel memory where the kernel logs messages from boot through runtime. `dmesg` reads and displays it. When it fills, oldest messages are overwritten (ring).

🔷 Core usage

```
# Show all kernel messages
dmesg

# Human readable timestamps (dmesg --human or -H)
dmesg -H

# Absolute timestamps (since epoch)
dmesg -T

# Follow (like tail -f)
dmesg -w
dmesg --follow

# Last N lines
dmesg | tail -50

# ── Filtering by level ──────────────────────────────────────────

# Log levels (0=emergency, 7=debug):
```

```
# 0 EMERG   — system is unusable
# 1 ALERT   — action must be taken immediately
# 2 CRIT    — critical conditions
# 3 ERR     — error conditions
# 4 WARNING — warning conditions
# 5 NOTICE  — normal but significant
# 6 INFO    — informational
# 7 DEBUG   — debug-level messages

# Show only errors and above
dmesg -l err
dmesg -l warn,err,crit    # Multiple levels
dmesg --level=err         # Long form

# Show only from specific facility
dmesg -f kern             # Kernel messages
dmesg -f daemon           # Daemon messages

# Kernel errors only (common for hardware issues)
dmesg -l err,crit,alert,emerg

# ── Filtering by content ──────────────────────────────────────

# OOM killer events
dmesg | grep -i "oom\|out of memory\|killed process"

# Disk errors
dmesg | grep -iE "I/O error|ata[0-9]|sd[a-
z]|nvme.*error|blk_update_request"

# Hardware errors
dmesg | grep -iE "MCE|machine check|hardware error|EDAC"

# Network issues
dmesg | grep -iE "eth[0-9]|ens[0-9]|link is (down|up)|NETDEV"

# Memory errors
dmesg | grep -iE "ECC|memory error|bad page|memory corruption"

# Boot messages only
dmesg | head -100   # First 100 lines = boot sequence

# Recent messages (last 5 minutes)
dmesg -T | grep "$(date -d '5 minutes ago' +'%b %d %H:%M')"

# Since last boot
dmesg --since "1 hour ago"     # If systemd journal integration
journalctl -k                  # Kernel messages from systemd journal
journalctl -k -b -1            # Previous boot's kernel messages
```

🔷 Critical patterns to recognize

```
# ── OOM Killer ──────────────────────────────────────────────────

dmesg | grep -A10 "Out of memory"
# [1234567.890] Out of memory: Kill process 4567 (java) score 892 or
sacrifice child
# [1234567.891] Killed process 4567 (java) total-vm:8192000kB, anon-
rss:7800000kB
# [1234567.892] oom_kill_process: oom_score_adj=0
#
# Key info:
# "score 892" — OOM score (higher = more likely to be killed)
# "total-vm:8192000kB" — 8GB virtual memory
# "anon-rss:7800000kB" — 7.6GB physical RAM used (this is what's freed)

# ── Hardware disk errors ────────────────────────────────────────

dmesg | grep "I/O error"
# [2345678.901] blk_update_request: I/O error, dev sda, sector 12345678
# [2345678.902] SCSI error: return code = 0x08000002
# [2345678.903] end_request: I/O error, dev sda, sector 12345678
#
# One or two errors: might be recoverable
# Repeated: disk is failing — check SMART immediately!
sudo smartctl -a /dev/sda | grep -E "Reallocated|Pending|Uncorrectable"

# ── Machine Check Exceptions (CPU/Memory hardware error) ────────

dmesg | grep -i "machine check\|MCE"
# [ 1234.567] mce: [Hardware Error]: Machine check events logged
# [ 1234.568] MEMORY CONTROLLER MS2(channel 0), transaction=data error
#
# MCE = hardware error — CPU, memory, bus
# Single MCE might be soft error (cosmic ray) — monitor
# Repeated MCE = hardware failure — replace DIMM/CPU

# ── Kernel warnings and bugs ────────────────────────────────────

dmesg | grep -i "BUG\|WARNING\|kernel NULL pointer"
# [12345.678] kernel BUG at /build/linux/mm/slub.c:3492!
# This is a kernel bug — report to vendor, consider kernel update

# ── Network drops ───────────────────────────────────────────────

dmesg | grep -i "NETDEV\|dropped\|tx timeout"
# [1234.567] eth0: Transmit timed out, status 0000, reset
# tx timeout = NIC or driver issue — check cables, replace NIC

# ── Filesystem errors ───────────────────────────────────────────

dmesg | grep -i "EXT4-fs error\|XFS.*error\|filesystem"
# [12345.678] EXT4-fs error (device sda1): ext4_find_entry:1455:
#             inode #2: comm mysqld: reading directory lblock 0
```

```
# Filesystem corruption — run fsck after clean unmount

# ── Thermal throttling ──────────────────────────────────────────

dmesg | grep -i "thermal\|throttl\|temperature"
# [ 1234.567] thermal thermal_zone0: critical temperature reached(103 C)
# [ 1234.568] CPU0: Package temperature above threshold, cpu clock
throttled
# CPU overheating → check cooling, airflow
```

🔷 Short crisp interview answer

> "`dmesg` reads the kernel ring buffer — the kernel's internal log. I use `dmesg -T` for human-readable timestamps and `dmesg -w` to follow live. Key things I look for: OOM killer events (`dmesg | grep 'Out of memory'` — shows which process was killed and how much memory it had); disk I/O errors (`dmesg | grep 'I/O error'` — early warning of disk failure); MCE events (hardware errors in CPU/memory); and filesystem errors that indicate corruption. `journalctl -k` shows the same kernel messages but with persistent storage across reboots."

7.8 OOM Killer — How Linux Kills Processes Under Memory Pressure

---

🔷 What the OOM Killer is

When the system runs out of memory AND cannot reclaim enough from caches AND cannot allocate from swap, the kernel's **Out-Of-Memory killer** activates: it selects and kills one or more processes to free enough memory to keep the system running.

🔷 How the OOM Killer works

```
Memory allocation request comes in:
         │
         ▼
Kernel tries in order:
1. Allocate from free pages ──────────── success → return
2. Reclaim page cache (drop clean pages) — success → return
3. Write dirty pages to disk, reclaim ─── success → return
4. Allocate from swap ──────────────────── success → return
5. None of the above work...
         │
         ▼
OOM condition detected
         │
         ▼
```

```
oom_kill_process() invoked
         │
         ▼
Calculate oom_score for every process:
  Score based on:
  - Memory used (anon RSS + swap) — higher = higher score
  - Memory used by children
  - Memory divided by time running (newer processes score higher)
  - Adjusted by oom_score_adj (-1000 to +1000)
         │
         ▼
Kill process with HIGHEST oom_score
(the biggest "badness" — kills the most to free the most)
```

🔷 OOM scores and tuning

```
# ── View OOM scores ──────────────────────────────────────────

# Every process has an oom_score (0-1000, higher = more likely to be
killed)
cat /proc/1234/oom_score
# 456

# View all processes sorted by OOM score (who's most at risk)
for pid in /proc/[0-9]*/oom_score; do
    score=$(cat "$pid" 2>/dev/null)
    pid_num=$(echo "$pid" | cut -d/ -f3)
    name=$(cat "/proc/$pid_num/comm" 2>/dev/null)
    echo "$score $pid_num $name"
done | sort -rn | head -20

# ── oom_score_adj — manual tuning ────────────────────────────

# oom_score_adj range: -1000 to +1000
# -1000 = NEVER kill this process (OOM exempt)
# +1000 = ALWAYS kill this process first
# 0     = default (no adjustment)

cat /proc/1234/oom_score_adj   # View current adjustment

# Protect a critical process from OOM (e.g., sshd — never lose SSH!)
echo -1000 > /proc/$(pgrep sshd)/oom_score_adj
# ⚠ Requires root

# Protect sshd permanently (add to systemd service):
# /etc/systemd/system/ssh.service.d/override.conf
# [Service]
# OOMScoreAdjust=-1000

# Make a process more likely to be killed first (sacrificial)
```

```
echo 1000 > /proc/$(pgrep unimportant_app)/oom_score_adj

# ── Protecting critical services ──────────────────────────────

# In systemd service file:
# [Service]
# OOMScoreAdjust=-900     # Very unlikely to be killed
# or
# OOMScoreAdjust=500      # Fairly likely to be killed (sacrifice this
one)

# Kubernetes equivalent:
# Pod QoS classes:
# Guaranteed  (requests==limits) → OOM score low → less likely killed
# Burstable   (requests<limits)  → medium OOM score
# BestEffort  (no requests/limits) → highest OOM score → killed first

# ── OOM killer logs ───────────────────────────────────────────

dmesg | grep -A20 "Out of memory"
# [1234567.890123] Out of memory: Kill process 4567 (java) score 892 or
sacrifice child
# [1234567.890456] Killed process 4567 (java) total-vm:8388608kB, anon-
rss:7340032kB, file-rss:4096kB, shmem-rss:0kB, UID:1000 pgtables:14336kB
oom_score_adj:0
# [1234567.890789] oom_reaper: reaped process 4567 (java), now anon-
rss:0kB, file-rss:0kB, shmem-rss:0kB

# Parse the log:
# "score 892"           → OOM badness score at time of kill
# "total-vm:8388608kB"  → 8GB virtual memory
# "anon-rss:7340032kB"  → 7.2GB physical RAM freed by killing this process
# "UID:1000"            → owned by user with UID 1000

# ── vm.overcommit settings ────────────────────────────────────

cat /proc/sys/vm/overcommit_memory
# 0 = heuristic overcommit (default): allow moderate overcommit
# 1 = always allow: never refuse malloc(), even if cannot honor
# 2 = never overcommit: refuse if total committed > swap + RAM*ratio

cat /proc/sys/vm/overcommit_ratio
# 50 = (with overcommit=2) allow committing 50% of RAM beyond swap

# View committed memory
grep "CommitLimit\|Committed_AS" /proc/meminfo
# CommitLimit:    18246120 kB  ← maximum memory the kernel will commit
# Committed_AS:   12345678 kB  ← total memory committed by all processes

# ── Preventing OOM ────────────────────────────────────────────

# Option 1: Add more RAM (obvious)

# Option 2: Add swap (gives more time, not a cure)
```

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Option 3: Tune vm.swappiness (prefer swap over dropping cache)
sudo sysctl vm.swappiness=60    # Default=60, lower=prefer cache over swap

# Option 4: Container memory limits (prevent one container killing the
host)
# In Docker: docker run --memory=2g myapp
# In Kubernetes: resources.limits.memory: 2Gi

# Option 5: Identify memory leaks
# Watch memory over time:
watch -n 5 'ps -eo pid,cmd,%mem,rss --sort=-rss | head -10'
# PID growing in RSS over time = memory leak
```

🔷 Short crisp interview answer

> "The OOM Killer activates when the kernel can't satisfy a memory allocation after exhausting free pages, page cache, and swap. It calculates an `oom_score` for every process based on memory consumption — bigger RSS means higher score, more likely to be killed. I tune this with `oom_score_adj`: I set `-1000` for critical processes like `sshd` (never kill) via systemd's `OOMScoreAdjust=-1000`. In Kubernetes, Guaranteed QoS pods get the lowest OOM scores. I diagnose OOM events with `dmesg | grep 'Out of memory'` which logs the killed process, its memory usage, and the score."

7.9 `systemtap` & `eBPF`/`bpftrace` — Production Tracing

🔷 The tracing landscape

```
Tracing tools evolution:
  SystemTap (2005):  kernel scripting, requires kernel debug headers
  ftrace (2008):     in-kernel function tracing, basis for perf
  eBPF (2014+):      safe in-kernel programs, JIT compiled, modern
standard
  bpftrace (2018):   high-level eBPF scripting language

Today's choice:
  Development/new:  bpftrace (easiest, most powerful)
  Older kernels:    SystemTap
  Low-level:        ftrace
  Production:       BCC tools (ready-made eBPF programs)
```

🔷 `bpftrace` — One-liners and scripts

```
# Install
sudo apt install bpftrace

# ── List available probes ──────────────────────────────────────────
sudo bpftrace -l 'tracepoint:syscalls:*'    # All syscall tracepoints
sudo bpftrace -l 'kprobe:tcp_*'             # All TCP kernel probes
sudo bpftrace -l '*open*'                   # Anything related to open

# ── One-liners ─────────────────────────────────────────────────────

# Count syscalls by process name
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count();
}'
# @[nginx]:  4523
# @[mysql]:  12345
# @[java]:   89012

# Trace all file opens systemwide
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_open,
                  tracepoint:syscalls:sys_enter_openat
                  { printf("%s opened %s\n", comm, str(args->filename));
}'

# Count file opens by process
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] =
count(); }'

# Measure read() latency distribution
sudo bpftrace -e '
tracepoint:syscalls:sys_enter_read { @start[tid] = nsecs; }
tracepoint:syscalls:sys_exit_read  /@start[tid]/
{
    @latency = hist(nsecs - @start[tid]);
    delete(@start[tid]);
}'
# @latency:
# [0, 1]       10  ▌
# [2, 4)       45  ▌▌▌▌▌▌▌
# [4, 8)      234  ▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌
# [8, 16)      89  ▌▌▌▌▌▌▌▌▌▌▌
# ...

# Trace new TCP connections with source/dest
sudo bpftrace -e '
kprobe:tcp_connect {
    $sk = (struct sock *)arg0;
    printf("%-16s → %s:%d\n",
        comm,
```

```
        ntop($sk->__sk_common.skc_daddr),
        $sk->__sk_common.skc_dport >> 8);
}'

# Count context switches by process
sudo bpftrace -e 'tracepoint:sched:sched_switch { @[prev->comm] = count();
}'

# Track block I/O latency
sudo bpftrace -e '
tracepoint:block:block_rq_issue { @start[args->dev, args->sector] = nsecs;
}
tracepoint:block:block_rq_complete
/@start[args->dev, args->sector]/
{
    @latency_us = hist((nsecs - @start[args->dev, args->sector]) / 1000);
    delete(@start[args->dev, args->sector]);
}'

# Show running processes every 1 second (poor man's top)
sudo bpftrace -e '
profile:hz:1 { @[comm] = count(); }
interval:s:1 { print(@); clear(@); }'

# ── bpftrace script file ──────────────────────────────────────────

cat /tmp/slow_reads.bt
#!/usr/bin/env bpftrace

// Find reads slower than 1ms
tracepoint:syscalls:sys_enter_read
{
    @start[tid] = nsecs;
    @fname[tid] = str(args->buf);
}

tracepoint:syscalls:sys_exit_read
/@start[tid]/
{
    $delta = nsecs - @start[tid];
    if ($delta > 1000000) {  // 1ms in nanoseconds
        printf("SLOW READ: %s took %d us\n", comm, $delta / 1000);
    }
    delete(@start[tid]);
    delete(@fname[tid]);
}

sudo bpftrace /tmp/slow_reads.bt
```

🔷 SystemTap — Older kernel support

```
# Install (requires kernel debug headers!)
sudo apt install systemtap linux-headers-$(uname -r)
sudo debuginfo-install kernel   # On RHEL/CentOS

# ── SystemTap scripts (stap) ──────────────────────────────────────

# Count syscalls per process
cat /tmp/syscall_count.stp
probe syscall.* {
    @count[execname()] <<< 1
}
probe end {
    foreach (name in @count- limit 10)
        printf("%s: %d\n", name, @count[name])
}

sudo stap /tmp/syscall_count.stp -T 10    # Run for 10 seconds

# Trace slow disk I/O
cat /tmp/slow_io.stp
probe ioblock.request {
    t = gettimeofday_us()
    if (rw == BIO_READ) reads[devname] = t
    else writes[devname] = t
}
probe ioblock.end {
    t = gettimeofday_us()
    elapsed = t - reads[devname]
    if (elapsed > 50000) {  // 50ms
        printf("SLOW IO: dev=%s elapsed=%dms size=%d\n",
            devname, elapsed/1000, size)
    }
}

sudo stap /tmp/slow_io.stp

# ── SystemTap vs bpftrace ─────────────────────────────────────────
# SystemTap: requires debug headers, compiles to kernel module, more
powerful
# bpftrace:  uses eBPF, no kernel module, safer, simpler syntax, modern
choice
# For new systems (kernel 4.4+): use bpftrace
# For older RHEL 6/7: use SystemTap
```

🔷 Short crisp interview answer

> "For production kernel tracing, I use `bpftrace` — it runs eBPF programs that are JIT-compiled and verified by the kernel (safe, cannot crash the system). One-liners are powerful: `bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'` counts file opens by process in real time. For block I/O latency, network connections, and slow syscalls I can write

> short scripts without kernel modules or reboots. SystemTap is the older alternative for kernels before 4.4 — it's more powerful but requires debug headers and compiles a kernel module."

7.10 Brendan Gregg's Tools — `execsnoop`, `opensnoop`, `biolatency`

🔷 The BCC/BPF tool collection

Brendan Gregg (Netflix) created a collection of ready-to-use eBPF-based tools in the **BCC** (BPF Compiler Collection) and **bpftrace** repositories. These are the tools that a modern SRE should know.

```
# Install BCC tools
sudo apt install bpfcc-tools
# Tools installed at: /sbin/*-bpfcc  or  /usr/sbin/*bpfcc

# Or via Python:
pip3 install bcc

# Clone the scripts
git clone https://github.com/brendangregg/perf-tools
git clone https://github.com/iovisor/bcc
```

🔷 `execsnoop` — Trace every exec() call

```
# What: Shows every new process execution in real time
# Why: Find short-lived processes that don't appear in ps/top
# Use: Debug scripts that spawn subprocesses, find unexpected processes

sudo execsnoop-bpfcc
# Or on newer systems:
sudo execsnoop

# Output:
# PCOMM            PID     PPID    RET ARGS
# nginx            12345   1       0   /usr/sbin/nginx -g daemon off;
# sh               12346   12345   0   /bin/sh -c ps aux
# ps               12347   12346   0   /usr/bin/ps aux
# grep             12348   12347   0   /usr/bin/grep --color=auto nginx
# sed              12349   12348   0   /usr/bin/sed s/foo/bar/

# PCOMM = parent command name
# RET   = return value (0=success, negative=error)
# ARGS  = full command with arguments

# Filter by process name
```

```
sudo execsnoop-bpfcc -n bash

# Filter by user
sudo execsnoop-bpfcc -u www-data

# ── Production use cases ──────────────────────────────────────────
# 1. Security: catch malicious process spawning
sudo execsnoop | grep -v "^PCOMM" | tee /var/log/exec_audit.log

# 2. Debug slow scripts (find which subprocesses are being called)
sudo execsnoop | grep -A1 "make\|gcc\|cc1"

# 3. Find what a service spawns (container escape detection)
sudo execsnoop -u nobody    # What is the web server running?

# 4. Trace cron job execution
sudo execsnoop | grep "cron\|crond"
```

🔷 `opensnoop` — Trace every file open

```
# What: Shows every file open (open/openat syscall) in real time
# Why: Find which files an app accesses, debug config loading, catch
secret reads

sudo opensnoop-bpfcc
# Or:
sudo opensnoop

# Output:
# PID    COMM               FD ERR PATH
# 12345  nginx               5   0 /etc/nginx/nginx.conf
# 12345  nginx               6   0 /etc/nginx/sites-enabled/myapp
# 12345  nginx              -1   2 /etc/nginx/ssl/missing.crt  ← ENOENT!
# 12346  java               12   0 /usr/lib/jvm/java-11/jre/lib/rt.jar
# 12347  python3            -1  13 /etc/shadow  ← EACCES!

# FD = file descriptor (-1 = failed to open)
# ERR = errno (0=success, 2=ENOENT no file, 13=EACCES permission denied)

# Filter by process
sudo opensnoop-bpfcc -p 1234

# Filter by filename
sudo opensnoop-bpfcc -n nginx

# Show only errors (FD=-1)
sudo opensnoop | awk '$3 == "-1"'

# Show only /etc access
sudo opensnoop | grep "/etc/"
```

```
# ── Production use cases ──────────────────────────────────────────
# 1. "App fails to start — what file is it looking for?"
sudo opensnoop-bpfcc -p $(pgrep myapp) | grep "ERR"

# 2. "What config files does this app read?"
sudo opensnoop-bpfcc -n myapp | grep -E "\.conf|\.yaml|\.json"

# 3. Security audit: what files does a process access?
sudo opensnoop-bpfcc -p 1234 | tee /tmp/file_access_audit.log

# 4. Find missing shared libraries
sudo opensnoop | awk '$3=="-1" && /\.so/'
```

🔷 `biolatency` — Block I/O latency histogram

```
# What: Shows disk I/O latency as a histogram (distribution)
# Why: Understand I/O latency — average hides outliers, histogram reveals
tail latency

sudo biolatency-bpfcc
# or: sudo biolatency

# Let it run for 30 seconds, then Ctrl+C:
# Tracing block device I/O... Hit Ctrl-C to end.
# ^C
#     usecs               : count     distribution
#         0 -> 1          : 0        |
|
#         2 -> 3          : 0        |
|
#         4 -> 7          : 0        |
|
#         8 -> 15         : 0        |
|
#        16 -> 31         : 0        |
|
#        32 -> 63         : 0        |
|
#        64 -> 127        : 1        |
|
#       128 -> 255        : 45       |***
|
#       256 -> 511        : 2345     |******************************
|
#       512 -> 1023       : 4567
|****************************************** |
#      1024 -> 2047       : 4890
|******************************************|
#      2048 -> 4095       : 1234     |**********
```

```
|
#       4096 -> 8191       : 123      |*
|
#       8192 -> 16383      : 45       |
|
#      16384 -> 32767      : 23       |
|  ← tail latency!
#      32768 -> 65535      : 5        |
|

# Reading the histogram:
# Most I/Os: 256us-4ms (good for SSD)
# Tail at 16-32ms: some I/Os are very slow (worth investigating)

# -D: show by disk device
sudo biolatency-bpfcc -D

# -F: show by flags (read vs write)
sudo biolatency-bpfcc -F

# -Q: include time in scheduler queue (total from submission to
completion)
sudo biolatency-bpfcc -Q

# ── Compare: iostat await vs biolatency ──────────────────────────
# iostat await = average latency (hides tail!)
# biolatency   = full distribution (reveals tail latency outliers)
# Example: await=2ms looks fine, but biolatency shows P99=50ms
# That 50ms is what causes your query timeouts!
```

🔷 More essential tools from the collection

```
# ── tcpconnect — trace outbound TCP connections ──────────────────

sudo tcpconnect-bpfcc
# PID    COMM         IP SADDR          DADDR          DPORT
# 12345  curl          4 10.0.0.5        93.184.216.34  443
# 12346  java          4 10.0.0.5        10.0.1.100     5432  ← DB
connection

# ── tcpaccept — trace inbound TCP connections ────────────────────

sudo tcpaccept-bpfcc
# PID    COMM         IP RADDR          RPORT LADDR     LPORT
# 1234   nginx         4 192.168.1.5     54321 10.0.0.5  443

# ── tcplife — complete TCP session lifetime ──────────────────────

sudo tcplife-bpfcc
# PID   COMM       LADDR      LPORT RADDR       RPORT TX_KB RX_KB MS
```

```
# 1234  nginx      10.0.0.5   80    1.2.3.4     54321  0.5   12.3   234.5
# Shows duration, bytes transferred — great for identifying slow sessions

# ── tcpretrans — TCP retransmissions (network quality indicator) ───────

sudo tcpretrans-bpfcc
# TIME     PID    IP LADDR:LPORT          T> RADDR:RPORT          STATE
# 14:32:01 1234    4 10.0.0.5:80          R> 1.2.3.4:54321
ESTABLISHED
# Retransmits = packet loss between you and the remote host

# ── profile — CPU profiler (like perf record) ──────────────────────────

sudo profile-bpfcc -F 99 30     # 99 Hz, 30 seconds
# Output: stack traces with counts — feed to flamegraph

sudo profile-bpfcc -F 99 30 | /opt/flamegraph/flamegraph.pl > profile.svg

# ── runqlat — CPU scheduler run queue latency ──────────────────────────

sudo runqlat-bpfcc
# Shows how long tasks wait in the run queue before getting CPU
# High run queue latency = CPU is saturated, processes waiting
#      usecs               : count     distribution
#          0 -> 1          : 2345
|************************************|  ← most tasks
#          2 -> 3          : 1234     |*******************
|
#         ...
#       1024 -> 2047       : 5        |
| ← some wait 1ms+

# ── cachestat — page cache hit/miss statistics ─────────────────────────

sudo cachestat-bpfcc 1
#    HITS   MISSES  DIRTIES  HITRATIO   BUFFERS_MB  CACHED_MB
#   45678     1234     5678    97.37%          98    9876
#   56789      234     4567    99.59%          98    9877
# HITRATIO near 100% = good, reads served from cache
# Many MISSES = lots of cold reads going to disk

# ── cachetop — per-process page cache stats ────────────────────────────

sudo cachetop-bpfcc 1
# PID    UID    CMD        HITS     MISSES   DIRTIES  READ_HIT%
WRITE_HIT%
# 1234   www    nginx       45678     1234     5678     97.37%     100.0%
# 5678   mysql  mysqld     234567     3456    12345     98.54%     100.0%

# ── slabtop — kernel slab cache usage (memory) ─────────────────────────

sudo slabtop
# Shows which kernel subsystems are using the most slab memory
# Useful: finding kernel memory leaks
```

```
# ── fileslower — slow file operations ──────────────────────────────

sudo fileslower-bpfcc 10     # Show operations slower than 10ms
# TIME(s)  COMM           TID    D BYTES   LAT(ms) FILENAME
# 0.001    mysqld        12345   R  4096     15.23  /var/lib/mysql/ibdata1
# 0.234    java          23456   W  8192     23.45  /var/log/app.log
```

🔷 Short crisp interview answer

> "Brendan Gregg's BCC tools are production-safe eBPF programs for specific observability tasks. `execsnoop` shows every process spawned in real time — invaluable for finding short-lived processes invisible to `ps`. `opensnoop` traces every file open with error codes — I use it to find what files an app can't open when it fails to start. `biolatency` shows disk I/O latency as a histogram — `iostat` gives averages that hide tail latency outliers, but `biolatency` reveals if 1% of I/Os take 50ms while the average is 2ms. `tcpretrans` spots packet loss in network connections, and `runqlat` shows CPU scheduling latency — how long tasks wait before they get CPU time."

🏆 Category 7 — Complete Mental Model

```
PERFORMANCE INVESTIGATION FLOWCHART
═══════════════════════════════════════

"System is slow" — where to start?
            │
            ▼
     ┌──────────────┐
     │  vmstat 2    │  ← 30-second check
     └──────────────┘
            │
     ┌──────▼──────────────────────────┐
     │ What does vmstat show?          │
     ├─────────────┬──────────────┬────┤
     │  r > nproc  │ b > 0, wa > 10% │ si/so>0 │
     └─────────────┴──────────────┴────┘
            │              │              │
      CPU bottleneck   I/O bottleneck   Memory pressure
            │              │              │
      mpstat -P ALL    iostat -hx 2    free -h; check
      (which core?)    iotop -o        available → swap
      perf top         (which process?)  dmesg OOM logs
      flamegraph       lsof -p PID      oom_score_adj

USE METHOD CHECKLIST (for each resource):
──────────────────────────────────────────
         │ Utilization        │ Saturation         │ Errors
```

```
──────────┼──────────────────────┼──────────────────────┼──────────────
CPU       │ mpstat %usr+%sys     │ vmstat r > nproc     │ dmesg MCE
Memory    │ free available       │ vmstat si/so > 0     │ dmesg EDAC
Disk      │ iostat %util         │ iostat aqu-sz > 1    │ smartctl -H
Network   │ sar -n DEV (MB/s)    │ retransmits          │ ethtool -S
──────────┼──────────────────────┼──────────────────────┼──────────────

TOOL QUICK REFERENCE:
═══════════════════════

Bird's-eye view:        vmstat 2
Per-CPU breakdown:      mpstat -P ALL 2
Memory detailed:        free -h + /proc/meminfo
Historical analysis:    sar -u / sar -r / sar -b (look back in time)
CPU hotspot:            perf top / sudo perf record + flamegraph
Load average meaning:   uptime → divide by nproc
Kernel messages:        dmesg -T | tail -50
OOM events:             dmesg | grep "Out of memory"
Process tracing:        bpftrace one-liners
New process tracking:   execsnoop-bpfcc
File open tracking:     opensnoop-bpfcc
Disk latency dist:      biolatency-bpfcc
TCP connections:        tcpconnect-bpfcc / tcpretrans-bpfcc
```

⚠ Master Gotcha List

#	Gotcha	Reality
1	`free` shows low free RAM	Check `available` — cache is reclaimable, low free is normal
2	Load average = CPU utilization	Load includes D-state I/O waiters — normalize by CPU count
3	vmstat first line is since boot	Ignore first line, watch subsequent lines for current state
4	High %util on SSD means saturated	SSD can parallel-process; check `aqu-sz` and `await` instead
5	swpd non-zero = active swapping	Only dangerous if `si`/`so` > 0 (actively moving pages)
6	OOM killer kills randomly	It calculates oom_score — biggest memory consumer first
7	perf benchmarks without --direct=1	Benchmarking page cache (RAM), not disk
8	`st` (steal) in vmstat ignored	Non-zero steal = hypervisor throttling your VM
9	load average lag	It's an EWMA — takes ~5 minutes to reflect changes
10	eBPF tools need root	Always `sudo bpftrace`, `sudo execsnoop`
11	mpstat shows avg, hides spikes	Use `mpstat -P ALL 1` to catch brief CPU spikes

#	Gotcha	Reality
12	iostat await = average, hides tail	Use `biolatency` for distribution — P99 may be 10× average

🔥 Top Interview Questions

Q1: System load is 25 but CPUs look fine in top. What's happening?

> Load average includes processes in D state (uninterruptible sleep — waiting for I/O), not just CPU usage. The system may have 25 processes blocked waiting for disk. Check `vmstat 1` — if `b` column is high and `wa` is high but `r` is low, it's an I/O bottleneck. Use `sudo iotop -o` to find the processes generating the I/O.

Q2: How do you investigate a performance regression that happened yesterday at 2PM?

> `sar` stores historical data. I'd run `sar -u -f /var/log/sysstat/sa<yesterday_date> | grep "02:"` to see CPU usage, `sar -r` for memory, `sar -b` for I/O, and `sar -n DEV` for network — all for that specific time window. This gives a complete picture of what the system was doing at that exact time without needing any foreknowledge of the incident.

Q3: How does the OOM killer choose which process to kill?

> It computes an `oom_score` (0–1000) for each process based primarily on memory consumption — processes using more RAM get higher scores. This score can be manually adjusted with `oom_score_adj` (-1000 to +1000). Setting `-1000` makes a process OOM-exempt. In practice I protect `sshd` (so I don't lose remote access) and critical databases. The OOM log in `dmesg` shows the killed process's score, RSS, and virtual memory — useful for understanding what triggered it.

Q4: What's the difference between `perf top` and a flamegraph?

> `perf top` shows a live ranked list of hot functions — good for immediately seeing where CPU is going. A flamegraph is a post-hoc visualization from `perf record -g` that shows the full call stack distribution: not just which function is hot, but which code path called it and how deep the call hierarchy goes. Flamegraphs reveal whether the hotspot is in your code or a library it calls, and they show the full context that led to a hot function.

Q5: How would you find all processes spawned by a web server in the last 5 minutes?

> `execsnoop-bpfcc` is the right tool — it uses eBPF to trace every `exec()` syscall with parent PID, child PID, and full command line. I'd run `sudo execsnoop-bpfcc -u www-data` to filter by the web server user. This catches short-lived processes (bash scripts, PHP scripts, etc.) that would never appear in `ps` because they finish before you check. It's also useful for security auditing — catching unexpected process spawning.

Q6: `free -h` shows the server is almost out of memory. What do you do first?

> I first check `available`, not `free`. If available is >1GB, the system is fine — Linux is just using RAM for file cache. The real danger signs are: available approaching zero, non-zero swap activity (`vmstat 1` showing `si`/`so` > 0), and `dmesg | grep 'Out of memory'` showing OOM kills. If genuinely

> under pressure, I check `ps -eo pid,cmd,rss --sort=-rss | head -10` to find the memory hog, then check if it's a leak with `watch -n 5 'ps -p <pid> -o pid,rss'`.

This document covers all 10 topics in Category 7: Performance & Observability — from vmstat fundamentals through eBPF tracing and Brendan Gregg's production toolset, with the USE Method framework for systematic investigation.